

Building Your First Neural Network with PyTorch

A Practical Guide to Image Classification from Scratch

Learn to implement, train, and evaluate a multi-layer perceptron for fashion item classification

Introduction

PyTorch has become the framework of choice for deep learning research and production. This tutorial walks through building a complete neural network pipeline from scratch—no prior deep learning experience required beyond basic PyTorch tensors and autograd concepts.

We'll construct an artificial neural network (ANN) to classify fashion items using the Fashion-MNIST dataset. By the end, you'll understand how to structure data, define architectures, implement training loops, and evaluate model performance.

1 The Problem: Fashion Item Classification

1.1 Why Fashion-MNIST?

Fashion-MNIST serves as an excellent entry point for neural network implementation. It's structurally identical to the classic MNIST digits dataset but replaces handwritten numbers with clothing items—making it more challenging and practically relevant.

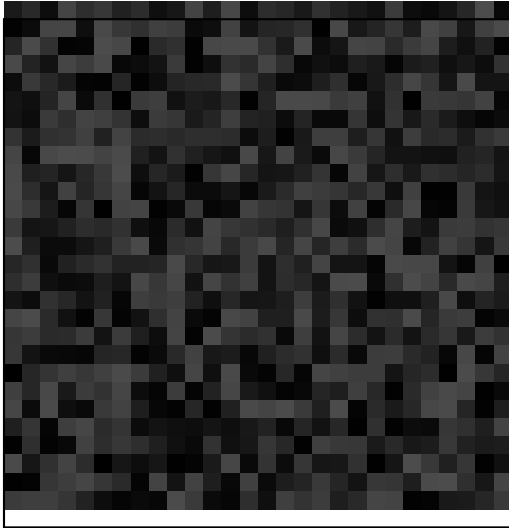
The dataset contains:

- 70,000 grayscale images (28×28 pixels)
- 10 clothing categories (T-shirts, trousers, pullovers, dresses, coats, sandals, shirts, sneakers, bags, ankle boots)
- CSV format: first column is the label (0–9), remaining 784 columns are pixel values (0–255)

For rapid experimentation, we'll use a 6,000-image subset. The full dataset requires GPU acceleration for efficient training—a topic we'll address in follow-up material.

1.2 Data Representation

Each image flattens into a 784-dimensional vector. A shoe might look conceptually like this:



→ [0.0, 0.0, 0.12, 0.85, ..., 0.03]

784 values normalized to [0,1]

2 Data Pipeline: From CSV to Batches

2.1 Loading and Preprocessing

Start with standard imports and reproducibility setup:

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 import torch
4 from torch.utils.data import Dataset, DataLoader
5 import torch.nn as nn
6 import torch.optim as optim
7 import matplotlib.pyplot as plt
8
9 torch.manual_seed(42) # Reproducibility
```

Listing 1: Imports and configuration

Load the data and inspect structure:

```
1 df = pd.read_csv('fmnist_small.csv')
2 print(df.head()) # 785 columns: label + 784 pixels
```

Listing 2: Loading the dataset

Extract features and labels, then split:

```
1 X = df.iloc[:, 1:].values # All pixel columns
2 y = df.iloc[:, 0].values # Label column
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.2, random_state=42
6 )
```

Listing 3: Feature extraction and train-test split

2.2 Normalization: Critical for Stable Training

Neural networks train poorly on raw pixel values (0–255). We normalize to [0,1]:

```
1 X_train = X_train / 255.0
2 X_test = X_test / 255.0
```

Listing 4: Feature scaling

Why Normalize?

Normalization prevents gradient saturation and ensures consistent gradient magnitudes across the network. Without it, training becomes unstable and convergence slows dramatically.

2.3 Custom Dataset Class

PyTorch requires data in a specific format. We extend `Dataset` to handle our numpy arrays:

```
1 class CustomDataset(Dataset):
2     def __init__(self, features, labels):
3         # Convert to tensors with appropriate dtypes
4         self.features = torch.tensor(features, dtype=torch.float32)
5         self.labels = torch.tensor(labels, dtype=torch.long)
6
7     def __len__(self):
8         return len(self.features)
9
10    def __getitem__(self, index):
11        return self.features[index], self.labels[index]
```

Listing 5: Custom Dataset implementation

Data Type Requirements

- Features must be `float32` for numerical computation
- Labels must be `long` (`int64`) for `CrossEntropyLoss`
- Wrong types trigger cryptic errors during training

Create dataset instances:

```
1 train_dataset = CustomDataset(X_train, y_train)
2 test_dataset = CustomDataset(X_test, y_test)
```

Listing 6: Dataset instantiation

2.4 DataLoader: Batching and Shuffling

`DataLoader` handles the mechanics of feeding data:

```
1 train_loader = DataLoader(
2     train_dataset,
3     batch_size=32,
4     shuffle=True           # Randomize order each epoch
5 )
6
7 test_loader = DataLoader(
```

```

8     test_dataset,
9     batch_size=32,
10    shuffle=False           # Consistent order for evaluation
11 )

```

Listing 7: DataLoader configuration

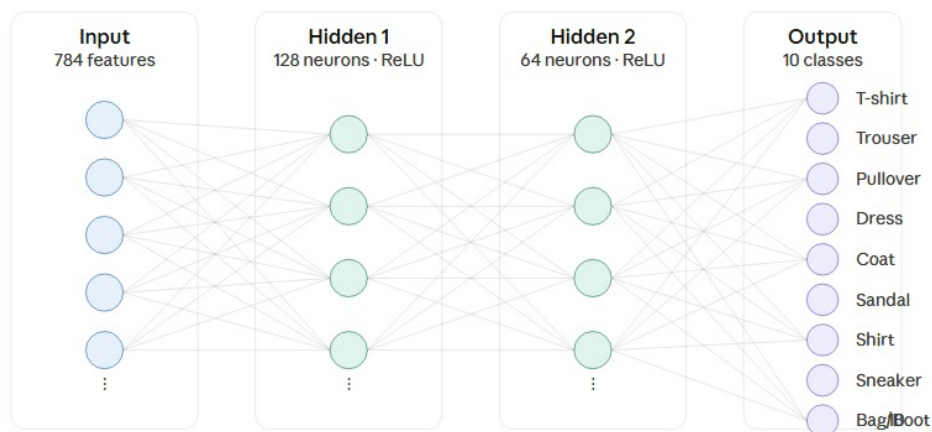
Batch Size Considerations

- **Small batches** (16–32): Noisy gradients, better generalization, slower training
- **Large batches** (256+): Stable gradients, risk of sharp minima, memory intensive
- **32–64**: Sweet spot for most applications

3 Neural Network Architecture

3.1 Network Design

Our multi-layer perceptron has this structure:



3.2 Mathematical Foundation

The forward pass computes:

$$\mathbf{z}_1 = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1 \quad (1)$$

$$\mathbf{a}_1 = \text{ReLU}(\mathbf{z}_1) = \max(0, \mathbf{z}_1) \quad (2)$$

$$\mathbf{z}_2 = \mathbf{W}_2 \mathbf{a}_1 + \mathbf{b}_2 \quad (3)$$

$$\mathbf{a}_2 = \text{ReLU}(\mathbf{z}_2) \quad (4)$$

$$\mathbf{z}_3 = \mathbf{W}_3 \mathbf{a}_2 + \mathbf{b}_3 \quad (5)$$

$$\hat{\mathbf{y}} = \text{Softmax}(\mathbf{z}_3) \quad (6)$$

3.3 PyTorch Implementation

Define the network by subclassing `nn.Module`:

```

1 class MyNN(nn.Module):
2     def __init__(self, num_features):
3         super().__init__() # Essential: initialize parent class
4
5         self.model = nn.Sequential(
6             nn.Linear(num_features, 128),
7             nn.ReLU(),
8             nn.Linear(128, 64),
9             nn.ReLU(),
10            nn.Linear(64, 10) # Output: 10 class logits
11        )
12
13    def forward(self, x):
14        return self.model(x)

```

Listing 8: Neural network class

Critical: Call `super().__init__()`

Forgetting `super().__init__()` prevents PyTorch from registering parameters. Your model will appear to train but won't learn—loss stays constant, accuracy remains at random chance. Always include this line.

No Explicit Softmax

PyTorch's `CrossEntropyLoss` combines `LogSoftmax` and `NLLLoss` internally. Pass raw logits (output of last linear layer) directly to the loss function. Adding an explicit `Softmax` layer creates incorrect gradients.

4 Training Setup

4.1 Hyperparameters

Configure the training process:

```

1 EPOCHS = 100
2 LEARNING_RATE = 0.1
3 BATCH_SIZE = 32
4
5 model = MyNN(X_train.shape[1]) # 784 input features
6 criterion = nn.CrossEntropyLoss()
7 optimizer = optim.SGD(model.parameters(), lr=LEARNING_RATE)

```

Listing 9: Training configuration

Parameter	Value	Rationale
Epochs	100	Sufficient for convergence on this dataset
Learning Rate	0.1	Aggressive updates for SGD
Batch Size	32	Balance between speed and gradient noise
Loss	CrossEntropy	Standard for multi-class classification
Optimizer	SGD	Baseline; Adam often converges faster

5 The Training Loop

5.1 Core Algorithm

Training involves four steps per batch:

1. **Forward pass:** Compute predictions
2. **Loss computation:** Measure prediction error
3. **Backward pass:** Calculate gradients via backpropagation
4. **Parameter update:** Adjust weights using gradients

5.2 Implementation

```
1 for epoch in range(EPOCHS):
2     total_epoch_loss = 0
3
4     for batch_features, batch_labels in train_loader:
5         # Forward pass
6         outputs = model(batch_features)
7
8         # Compute loss
9         loss = criterion(outputs, batch_labels)
10
11        # Backward pass
12        optimizer.zero_grad() # Clear previous gradients
13        loss.backward() # Compute new gradients
14
15        # Update parameters
16        optimizer.step()
17
18        total_epoch_loss += loss.item()
19
20    # Average loss per batch
21    avg_loss = total_epoch_loss / len(train_loader)
22    print(f'Epoch: {epoch + 1}, Loss: {avg_loss:.4f}')
```

Listing 10: Training loop implementation

Gradient Accumulation Pitfall

`optimizer.zero_grad()` must precede `loss.backward()`. PyTorch accumulates gradients by default; without clearing, gradients from previous batches corrupt the current update. This is a common source of training instability.

5.3 Expected Training Dynamics

You'll observe output similar to:

```
Epoch: 1, Loss: 1.3216
Epoch: 2, Loss: 0.7793
Epoch: 3, Loss: 0.6428
...
Epoch: 50, Loss: 0.1008
Epoch: 100, Loss: 0.0728
```

Loss decreasing from ~ 1.3 to ~ 0.07 indicates successful training. If loss plateaus early or oscillates, adjust learning rate or check for bugs.

6 Evaluation and Accuracy

6.1 Setting Evaluation Mode

Before testing, switch the model to evaluation mode:

```
1 model.eval()
```

Listing 11: Evaluation mode

This disables dropout and uses running batch normalization statistics—critical for consistent inference even if not used in this simple architecture.

6.2 Manual Accuracy Calculation

We compute accuracy without external libraries to understand the mechanics:

```
1 total = 0
2 correct = 0
3
4 with torch.no_grad(): # Disable gradient computation
5     for batch_features, batch_labels in test_loader:
6         outputs = model(batch_features)
7
8         # Get predicted class (index of max logit)
9         _, predicted = torch.max(outputs, dim=1)
10
11         total += batch_labels.size(0)
12         correct += (predicted == batch_labels).sum().item()
13
14 accuracy = correct / total
15 print(f'Test Accuracy: {accuracy:.2%}')
```

Listing 12: Evaluation implementation

Understanding torch.max()

`torch.max(outputs, dim=1)` returns:

- Maximum values across dimension 1 (ignored with `_`)
- Indices of maximum values (predicted class labels)

For a batch of 32 samples with 10 classes, `outputs` has shape (32, 10). `predicted` has shape (32,) containing class indices 0–9.

6.3 Interpreting Results

Expected output: Test Accuracy: 83.25%

This means 83 of 100 test images are correctly classified. For 10 balanced classes, random guessing achieves 10%—our network has learned meaningful patterns.

7 Improvement Roadmap

83% accuracy establishes a baseline. Several avenues exist for improvement:

7.1 Immediate Wins

- **Full dataset:** Train on all 70,000 images instead of 6,000
- **Longer training:** 200–500 epochs with learning rate decay
- **Optimizer change:** Adam often outperforms SGD on this task

7.2 Architectural Enhancements

- **Dropout:** Add `nn.Dropout(0.5)` between layers to prevent overfitting
- **Batch normalization:** Stabilize training and allow higher learning rates
- **Wider/deeper networks:** 256 or 512 neurons per layer, or add a third hidden layer

7.3 Advanced Techniques

- **Convolutional layers:** Replace fully-connected layers with CNNs for spatial feature extraction
- **Data augmentation:** Random rotations, crops, and flips to increase effective dataset size
- **Learning rate scheduling:** Reduce LR when loss plateaus
- **Early stopping:** Halt training when validation loss stops improving

8 Complete Code

Here's the consolidated implementation for reference:

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 import torch
4 from torch.utils.data import Dataset, DataLoader
5 import torch.nn as nn
6 import torch.optim as optim
7
8 # Configuration
9 torch.manual_seed(42)
10 EPOCHS, LR, BATCH_SIZE = 100, 0.1, 32
11
12 # Data preparation
13 df = pd.read_csv('fmnist_small.csv')
14 X, y = df.iloc[:, 1:].values, df.iloc[:, 0].values
15 X_train, X_test, y_train, y_test = train_test_split(X, y,
16                                                    test_size=0.2, random_state=42)
17 X_train, X_test = X_train / 255.0, X_test / 255.0
18
19 # Dataset and DataLoader
20 class CustomDataset(Dataset):
21     def __init__(self, X, y):
```



```

21         self.features = torch.tensor(X, dtype=torch.float32)
22         self.labels = torch.tensor(y, dtype=torch.long)
23     def __len__(self): return len(self.features)
24     def __getitem__(self, idx): return self.features[idx],
        self.labels[idx]
25
26 train_loader = DataLoader(CustomDataset(X_train, y_train),
        batch_size=BATCH_SIZE, shuffle=True)
27 test_loader = DataLoader(CustomDataset(X_test, y_test),
        batch_size=BATCH_SIZE, shuffle=False)
28
29 # Model definition
30 class MyNN(nn.Module):
31     def __init__(self, n_features):
32         super().__init__()
33         self.model = nn.Sequential(
34             nn.Linear(n_features, 128), nn.ReLU(),
35             nn.Linear(128, 64), nn.ReLU(),
36             nn.Linear(64, 10)
37         )
38     def forward(self, x): return self.model(x)
39
40 # Training
41 model = MyNN(X_train.shape[1])
42 criterion, optimizer = nn.CrossEntropyLoss(),
    optim.SGD(model.parameters(), lr=LR)
43
44 for epoch in range(EPOCHS):
45     epoch_loss = 0
46     for features, labels in train_loader:
47         loss = criterion(model(features), labels)
48         optimizer.zero_grad()
49         loss.backward()
50         optimizer.step()
51         epoch_loss += loss.item()
52     print(f'Epoch {epoch+1}: {epoch_loss/len(train_loader):.4f}')
53
54 # Evaluation
55 model.eval()
56 correct = total = 0
57 with torch.no_grad():
58     for features, labels in test_loader:
59         _, predicted = torch.max(model(features), 1)
60         total += labels.size(0)
61         correct += (predicted == labels).sum().item()
62 print(f'Accuracy: {correct/total:.2%}')

```

Listing 13: Complete neural network implementation

Key Takeaways

- **Data preparation** is half the battle: proper normalization and tensor conversion prevent training failures
- **Architecture matters**: Even simple MLPs require careful layer sizing and activation selection

- **The training loop** follows a strict pattern: forward, loss, zero-grad, backward, step
- **Evaluation requires care:** Set `eval()` mode, disable gradients, and understand output shapes
- **Baselines are essential:** 83% gives you a reference point for measuring improvements

This foundation prepares you for more complex architectures: convolutional networks for computer vision, recurrent networks for sequences, and transformers for natural language processing. The patterns established here—data loading, model definition, training loops, and evaluation—remain consistent across all deep learning applications.